Yankı — NPC ve Sonuç Motoru

Proje Planı v2 (28 Ekim 2026)

Çalışma adı: "Yankı". Oyuncunun her kararı NPC'lerde ve dünyada yankılanır. İsim sonradan değişebilir.

v2, v1'in benzer akademik, ticari ve açık kaynak sistemlerle karşılaştırılmasıyla hazırlandı. Kaynaklar ve gerekçeler: `reports/NPC sonuç motoru benzer sistemler.md`.

0. v1 → v2: Ne değişti ve neden

Temel kararlar aynı kalıyor: kural + utility AI, şablon diyalog + yapay zeka arayüzü, olay kaynaklı çekirdek, 10–50 derin NPC, motordan bağımsız C# ve oyunun ayrı, sonraki bir proje olması. Değişiklikler bu kararları inceltiyor, tersine çevirmiyor.

#	Ne değişti	Neden (kısaca)
1	Hedef: `netstandard2.1;net10.0`	Unity yalnızca netstandard2.1 kabul ediyor. .NET 8 ve 9'un desteği 10 Kasım 2026'da bitiyor, .NET 10 güncel LTS.
2	Kurulumsuz tarayıcı demosu (Blazor WebAssembly). Demo ve hata ayıklama ekranı tek uygulamada.	İşe alımcı 30 saniye bakıyor. "Oyunu olmayan motor" algısını kırıyor. Godot C# web'e export edemiyor.
3	Sonuç kuralları ve diyalog için tek **kriter dili** (Valve tarzı ağırlıklı kriterler)	Tek ayrıştırıcı, tek doğrulayıcı, tek kapsama aracı yetiyor.
4	İlişki: 3 kalıcı eksen + **zamanlı fikir modifier'ları** (süre, birikme sınırı, azalan etki)	RimWorld modeli. "Neden kızgın?" sorusunun cevabı modifier listesinde.
5	Tutarlılık mekanizması: **kişiliğe aykırılık stresi** ve eşikte kriz olayı	CK3 ve Dwarf Fortress. Kişilik davranışı yasaklamıyor, fiyatlandırıyor.
6	İnanç = **kanıt listesi**. Güven bu listeden hesaplanıyor.	Talk of the Town. "Kimden duydu?" birinci sınıf bir sorgu oluyor.
7	Dedikodu sohbet başına **top-n konu** ile sınırlı	Bilgi patlamasına karşı somut fren
8	Seçim varsayılanı: ağırlıklı **top-3**, anahtarlı RNG	Sims'te argmax robotik duruyor. Anahtarlı RNG sıradan bağımsız determinizm sağlıyor.
9	Olaylarda `NedenOlaylar[]` + `KuralId`	Nedensellik defteri. "Neden?" zinciri ve kapsama raporu buradan çıkıyor.
10	Kaskad sınırları, cooldown'lar, **korumalı bayraklar**, yay değişkenleri	Dwarf Fortress'ın sarmalları, Oblivion'ın görev kıran NPC'leri
11	Takılabilir `IActionSelector`	GOAP ve BT kütüphaneleriyle yarışmak yerine onlara kapı açıyor
12	Test: golden replay hash'leri, CsCheck, BenchmarkDotNet	Kanıtlanabilir determinizm ve performans
13	Değerlendirme: tek seçim farkı replay raporu, ablasyon, küçük kör playtest	"Seçimler önemli" iddiasını sayıyla desteklemek
14	Minimal yazım şeması, doğrulayıcı kuralları, kural kapsama aracı	Bütün benzer sistemlerin bir numaralı maliyeti yazım ve ayar
15	Repo vitrini: README iskeleti 1. haftada, docs sitesi, `ai-usage.md`, devlog, NuGet	Portfolyo hedefi
16	Takvim: 4. haftada ince dikey dilim. Snapshot, Yarn köprüsü, Unity örneği vb. "v0.2 sonrası" listesine taşındı. Yılbaşı ve final haftaları hafifletildi.	v1 150–180 saate sığmıyordu, tamponu da yoktu

1. Özet

Yankı, oyundan bağımsız bir **C# NPC ve sonuç motoru**. Amacı oyunlara şunları kazandırmak:

- tutarlı karakterlere sahip NPC'ler,
- doğal ilerleyen olaylar,
- gerçekten hissedilen seçimler,
- tekrar oynandığında yeni diyaloglar ve ilerleyişler.

Motor bir kütüphane olarak tasarlanır ve `netstandard2.1` ile `net10.0` hedeflerine derlenir. Böylece Unity, Godot (C#), MonoGame ve sunucu projelerine takılabilir. Motoru göstermek için tarayıcıda kurulumsuz oynanan küçük bir demo ("Kasaba Hanı") ve aynı sayfada NPC zihinlerini canlı gösteren bir panel hazırlanır.

Temel fikir: **hikaye önceden yazılmaz, olaylardan doğar.** Oyuncunun kararları, zamanlaması ve NPC'lerle hangi sırayla

konuştuğu, NPC'lerin bildiklerini, inançlarını, ilişkilerini ve kararlarını gerçekten değiştirir. Her değişikliğin nedeni olay günlüğünden geriye doğru izlenebilir.

Konumlandırma (README'deki tek cümle): Anlatı oyunları için deterministik, tekrar oynatılabilir NPC hafızası ve sonuç sistemi; Unity, Godot ve sunucuda çalışan açık bir C# kütüphanesi. Love/Hate (ticari, sadece Unity) ve Ensemble (JS, araştırma kodu) ile karşılaştırma tablosu README'de yer alır.

Proje, Yusuf'un mezuniyet sonrası iş arama sürecinde GitHub profilinde yetkinliğini gösterecek ana portfolyo projesidir. Kodun büyük kısmı Claude Code ile yazılır. Yusuf sistem tasarımını, kuralları, dengeyi, içerik metinlerini ve test senaryolarını belirler; bunu repoda açıkça belgeler.

2. Çözülen problem

Oyunlardaki seçim sistemlerinin çoğu, önceden yazılmış **dallanan diyalog ağaçlarına** dayanır. Ink ve Yarn Spinner gibi araçlar bu yaklaşımın standart araçlarıdır. Bu yaklaşımın iki zayıf noktası var:

1. **Seçim illüzyonu.** Dallar katlanarak çoğaldığı için yazarlar dalları kısa sürede tekrar ana hikayeye bağlar. Oyuncu seçimin hiçbir şeyi değiştirmediğini fark eder. Telltale oyunlarının "bunu hatırlayacak" yazısı bunun en bilinen örneği. Eleştirilerin ortak noktası, karakterlerin geçmiş seçimleri hatırlamaması.
2. **Statik NPC'ler.** NPC'ler oyuncu onlarla konuşmadığı sürece donmuş halde bekler. Birbirleriyle bilgi paylaşmazlar. Konuşma sırası ve zamanlama hiçbir şeyi değiştirmez.

Yankı bu iki soruna şu yaklaşımla cevap verir: dalları elle yazmak yerine, **NPC zihinlerini ve dünyayı simüle eden kurallar** yazılır.

Yankı, Ink ve Yarn Spinner'ın **rakibi değil, tamamlayıcısıdır**. Onlar sahnenin metnini taşır, Yankı ise kimin neyi bildiğini ve neyi hatırladığını. v0.2'de bir Yarn Spinner köprüsü planlanıyor (bkz. 16).

Diğer sistemlerin bıraktığı açıklar şunlar:

- Açık kaynak, motordan bağımsız ve bakımı süren bir C# kütüphanesi yok. Sosyal durum, bilgi yayılımı ve utility kararını birleştiren en yakın ürün Love/Hate; o da ücretli ve sadece Unity'de.
- Akademik motorlar (Ensemble, Talk of the Town, Neighborly) JS ya da Python'da yazılmış ve bir oyuna gömülemiyor.
- LLM tabanlı NPC'ler pahalı, deterministik değil ve olay uydurabiliyor.

3. Hedefler ve başarı kriterleri

Hedefler

- Bir NPC'nin kişiliği tutarlı kalır. Bir iyilik, açgözlü bir NPC'yi bonkör yapmaz; sadece o kişiye karşı bir minnet modifier'ı ekler. Kişiliğe aykırı davranış mümkündür ama stres biriktirir.
- Aynı seçimler farklı sırayla yapıldığında ölçülebilir şekilde farklı sonuçlar çıkar.
- NPC'ler oyuncu bakmıyorken de kendi hedeflerine göre hareket eder ve birbirleriyle bilgi paylaşır. Bunu yaparken korumalı varlıkları ve hikaye yayını bozmazlar.
- Aynı başlangıçla, farklı seed ya da farklı kararlarla oynandığında farklı diyaloglar ve gidişatlar oluşur.
- Aynı seed ve aynı eylem dizisi her zaman aynı sonucu verir (determinizm). Bu, masaüstü .NET 10 ile tarayıcıdaki WebAssembly arasında da geçerlidir.
- Her NPC kararı ve her zihin değişikliği için "neden?" sorusu olay zincirinden cevaplanabilir.

Ölçülebilir başarı kriterleri (24 Ocak 2027)

- 50 NPC'lik senaryoda tick başına ortalama süre 5 ms'nin altında. BenchmarkDotNet tablosu README'de; donanım ve .NET sürümü belirtilmiş.
- CI'da en az 3 golden replay: kayıtlı seed ve girdi günlüğü tekrar oynatıldığında son durum hash'i birebir tutuyor. Biri tarayıcı (WASM) çıktısıyla karşılaştırılıyor.
- En az 3 "sıra etkisi" senaryosu otomatik testle kanıtlanmış.
- En az 1 **tek seçim farkı raporu**. Bir oyuncu kararı farklı olduğunda N tick sonra kaç olayın, inancın ve diyalog satırının ayrıştığı tabloyla README'de gösteriliyor. Dedikodu ve stres kapatılarak ablasyon yapılmış.
- Demo senaryosunda 20 seed'lik toplu koşuda her sonuç kuralı en az bir kez tetikleniyor. Hiçbir kural tetiklenmelerin %30'undan fazlasını tek başına almıyor. Kullanılmayan kişilik özelliği yok.
- 3–5 kişiyle küçük bir kör playtest yapılmış, notları devlog'da.
- Tarayıcıda oynanabilir demo GitHub Pages'te; linki README'nin ilk ekranında.
- README, mimari dokümanı, ADR'ler, `docs/ai-usage.md`, docs sitesi, CI, coverage rozeti, NuGet'te `v0.1.0`.

Kapsam dışı (bilinçli olarak)

- Binlerce NPC'lik kalabalık simülasyonu (hedef 10–50 derin NPC).
- Yapay zekanın NPC kararlarını vermesi. Yapay zeka en fazla metni seslendirir (`IDialogueRenderer`).
- Grafik, animasyon ve yol bulma (bunlar oyunun işi).
- Doğal dil girdisi. Oyuncu menüden eylem ve konu seçer.
- GOAP veya davranış ağacı planlayıcısı yazmak. Bunun yerine `IActionSelector` arayüzü var.
- Tam bir ticari oyun. Dedektif oyunu ayrı ve sonraki bir proje.

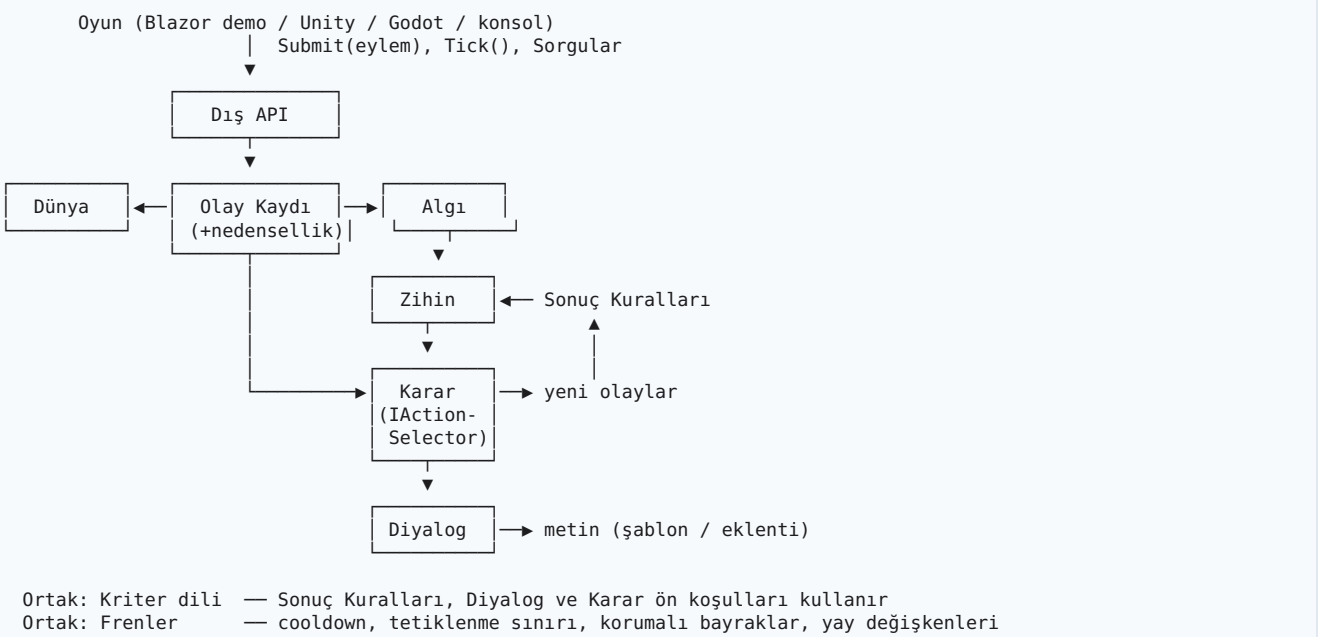
4. Şu ana kadar alınan kararlar

#	Karar	Seçilen	Gerekçe
1	Proje türü	Önce motor, oyun sonra; motor küçük bir tarayıcı demosuyla birlikte yayınlanır	İki ayrı proje çıkar. Demo, "oyunu olmayan motor" algısını kırar.
2	NPC karar mekanizması	Kural + utility AI, takılabilir `IActionSelector` arkasında	Deterministik, test edilebilir, ucuz; tutarlı karakterler. GOAP/BT ile yarışmaz, onlara açık.
3	Diyalog	Şablonlu üretim (ortak kriter diliyle seçim) + yapay zeka eklentisi için arayüz	Çevrimdışı ve tekrar üretilebilir. Yapay zeka sadece tek yönlü seslendirir.
4	Çekirdek mimari	Olay kaynaklı (event-sourced), nedensellik defteriyle	Sıra etkisi, tekrar oynatma, kayıt/yükleme ve "neden" sorusu kendiliğinden çözülür
5	Ölçek	10–50 derin NPC	Derinlik, kalabalıktan önemli
6	Dil / biçim	C#, oyun motorundan bağımsız kütüphane, `netstandard2.1;net10.0`	Yusuf'un C# bilgisi; Unity, Godot ve .NET uyumu
7	Demo teknolojisi	Blazor WebAssembly (demo + zihin paneli tek uygulama), GitHub Pages	Saf .NET, çekirdeği doğrudan çalıştırır, kurulum istemez. Godot C# web'e çıkamaz, Unity WebGL ikinci bir motor öğrenme yükü getirir.
8	Lisans	MIT	Love/Hate'e karşı açık alternatif konumu; ekosistemdeki norm
9	Kural ve diyalog dili	Tek ortak kriter dili (JSON)	Tek araç zinciri, daha az yazım yükü

Unity WebGL'in reddedilme gerekçesi şu: Türk mobil stüdyoları Unity tecrübesini artı sayıyor. Ama 12 haftada hem Unity öğrenmek hem debugger'ı Unity'de ayrıca yazmak takvimi taşırır. Unity entegrasyonu, v0.2'nin ilk maddesi olarak küçük bir örnek projeyle gösterilecek. Karar ADR-006'da kayıtlı.

5. Mimari

Motor, her biri tek bir işten sorumlu sekiz modülden ve bunların ortak kullandığı bir **kriter dilinden** oluşur. Modüller birbirleriyle sadece tanımlı arayüzler üzerinden konuşur.



5.1 Olay Kaydı (Event Log)

- Tek doğruluk kaynağı. Sadece sonuna ekleme yapılabilir, kayıtlar değiştirilemez.
- Her olay şunları taşır: `Id`, `Tick`, `Sıra`, `Tür`, `Fail`, `Hedef`, `Mekan`, `Yük`, `NedenOlaylar[]` (bu olaya yol açan olay kimlikleri) ve `KuralId` (olayı üreten sonuç kuralı ya da eylem tanımı; oyuncu eylemlerinde `player`).
- "Bu NPC neden buna karar verdi?" sorusu `NedenOlaylar` zinciri geriye doğru takip edilerek cevaplanır. `KuralId` ise kural kapsama raporunu besler.
- **Kayıt/yükleme v0.1'de:** seed + başlangıç senaryosu + oyuncu eylem günlüğü. Yükleme, günlüğü baştan oynatmaktır. 50 NPC ve bir haftalık oyun içi süre için bu yeterince hızlı (benchmark'la doğrulanacak). Anlık görüntü (snapshot) v0.2'ye kaldı.
- Her tick sonunda **durum hash'i** alınır. Değerler sabit hassasiyete yuvarlanır, koleksiyonlar sıralı gezilir. İki koşunun nerede ayrıştığını bulmak için kullanılır.

5.2 Dünya

- NPC'ler, mekanlar (bağlantı grafiği), eşyalar ve **gerçekler** (fact).
- Bir gerçek, dünyada nesnel olarak doğru olan bir önermedir. Örnek: `borçlu(Mehmet, Ayşe, 500)`.
- **Tek değerli gerçekler** `!` ile işaretlenir. Örnek: `konum!(Ali, han)`. Yeni değer yazılınca eski değer otomatik düşer, kurallarda elle silmek gerekmez (Versu'daki tek değerli slotlar).
- Varlıklar **korumalı** işaretlenebilir (`korumalı: [öldürülemez, çalınamaz, mekandan_ayrılamaz]`). Bkz. 5.10.
- Kimin ne bildiği burada tutulmaz, Zihin modülünde tutulur.

5.3 Algı

- **Doğrudan algı:** Bir olay olduğunda, aynı mekanda bulunan ve olayın **görünürlük** seviyesini (`açık`, `sessiz`, `gizli`) karşılayan NPC'ler olayı algılar ve inançlarına `gördü` kanıtı eklenir.
- **Bilgi yayılımı (dedikodu):** NPC'ler karar modülünde "anlat" eylemini seçtiklerinde bildiklerini aktarır. Karşı tarafa `duydu(kaynak)` kanıtı eklenir. Kanıtın gücü, anlatana duyulan güvenle ve kanıtın kendi gücüyle çarpılır, yani aktarımda düşer.
- **Konu sınırı:** Bir sohbette konuşulabilecek konu sayısı `n = 1 + ⌊sosyallik × 2⌋ (+1 yakın ilişkide)` ile sınırlıdır. Konular dikkat çekicilik puanına göre seçilir: önem × yenilik × konuşan ve dinleyenle ilgisi. Bu, Talk of the Town'ın "en dikkat çekici n varlık" freni.
- Yalan söyleyen bir NPC yanlış bir önerme aktarabilir. Yalanı söyleyen bunu bilir; bu bilgi hafızasında `yalan_söyledim(kime, ne)` olarak tutulur.

5.4 Zihin

Her NPC'nin zihni farklı hızlarda değişen katmanlardan oluşur:

Katman	Örnek alanlar	Değişim hızı
Kişilik	açgözlülük, dürüstlük, cesaret, sosyallik, kin tutma, merhamet (0–1 arası)	Sabit (v0.1'de kayma yok)
İlişki: kalıcı eksenler (kişi başına)	güven, sevgi, korku (−100..100, sınırlı)	Yavaş; sadece büyük olaylarla
İlişki: fikir modifier'ları	`minnet(+20, 14 gün)`, `hakaret_etti(−15, 20 gün)`; birikme sınırı ve ×0.9 azalan etki	Olayla gelir, süresi dolunca düşer
Ruh hali	tek baskın duygu: tür + şiddet + hedef + neden olan olay; bir önceki duygu da tutulur	Hızlı yükselir, zamanla söner
Stres	0–100, kişiliğe aykırı eylemlerle artar	Yavaş söner; eşikte kriz
Hafıza	olay referansı + önem; önemsizler zamanla silinir, önemliler kalıcıdır	Olaya bağlı
İnançlar	önerme + **kanıt listesi** (tür: gördü / duydu / çıkarım; kaynak, tick, güç)	Algı ve dedikoduyla
Hedefler	kalıcı ya da duruma bağlı amaçlar	Kurallarla

Fikir (opinion) hesabı: `fikir(A→B) = kalıcı eksenlerin ağırlıklı toplamı + Σ aktif modifier'lar`. Eşik bantları kriterlerde kullanılır: `düşman < −40 ≤ soğuk < −10 ≤ nötr ≤ 10 < yakın ≤ 40 < dost`. Her modifier, onu doğuran olayın kimliğini taşır. Böylece hata ayıklama ekranı "Can sana −22: Ayşe'den duyduğu 'aileme iftira' (tick 41, 18 gün kaldı)" gibi satırlar gösterir.

İnanç güveni: `güven = 1 − Π(1 − kanıt_gücü)`, destekleyen kanıtlar üzerinden hesaplanır. Çelişen iki önerme varsa birikmiş kanıtı güçlü olan "inanılan" sayılır. Kanıtlar silinmez, sadece zayıflar. "Bunu kimden duydu?" sorusu kanıt listesinden okunur.

Tutarlılık ilkesi (v2): İlişki ve ruh hali kişiliği ezemez. Bu ilke iki mekanizmayla uygulanır:

1. **Puan tarafında:** Kişilik çarpanı karar formülünde en geniş aralığa sahiptir (bkz. 5.5). Minnet, sadece o kişiye yönelik

eylemlerin puanını etkiler.

2. **Maliyet tarafında (stres):** Her eylem tanımı hangi özelliklerle çeliştiğini bildirir. Örnek: `yalan söyle` → `dürüstlük`. Böyle bir eylem seçilirse `stres += taban × özellik_değeri` uygulanır. Stres 50'yi ve 80'i geçince kişiliğe göre seçilen bir **kriz** olayı tetiklenir ve stres düşer. Örnekler: dürüst biri itiraf eder, kinci biri patlar, sosyal biri birine dert yanar. Kişilik ağırlıkları hiç değişmez. Kişiliğe aykırı davranış mümkündür ama pahalıdır ve hikaye üretir (CK3 / Dwarf Fortress).

5.5 Karar (Utility AI)

- Eylemler JSON ile tanımlanır. Her eylemin ön koşulları (kriter dili, `zorunlu` kriterler), puanlama ölçütleri (consideration), **en az bir etkisi** ve isteğe bağlı `çeliştiği_özellikler` alanı vardır.
- Örnek eylemler: anlat, yalan söyle, yardım et, iste, reddet, kaçın, tehdit et, hediye ver, bir yere git, sakla.
- **Puan formülü:** `puan = taban × Π(ölçüt_eğrisi(girdi)) × kişilik_çarpanı × fikir_çarpanı × ruh_hali_çarpanı` Kişilik çarpanı 0.25–2.0 aralığında, fikir çarpanı 0.6–1.5 aralığında, ruh hali çarpanı 0.8–1.25 aralığında. Bu aralıklar tutarlılık ilkesinin sayısal karşılığı ve ADR'de gerekçelendirilir.
- Ölçütler eğri tipleriyle tanımlanır: doğrusal, üstel, lojistik ve basamak.
- **Seçim:** Varsayılan, puanı en yüksek eylemin %70'ini geçen ilk 3 aday arasından ağırlıklı seçim. N ve eşik ayarlanabilir; `N=1` deterministik argmax demek. RNG, `(seed, tick, npcId, "karar")` anahtarından türetilir. Böylece NPC işleme sırası değişse bile her NPC'nin zarı aynı kalır.
- Her karar bir **karar izi** üretir: hangi eylemler ne puan aldı, hangi ölçüt ne katkı yaptı, hangisi seçildi. Bu iz, hata ayıklama ekranında gösterilir ve diyaloğa "gerekçe" olarak verilebilir (F.E.A.R.'ın kararı sonradan seslendiren diyaloğu gibi).
- **`IActionSelector` arayüzü:** `Seç(npc, adaylar, bağlam) → (eylem, iz)`. Varsayılan uygulama `UtilitySelector`. Bir oyun bunun yerine kendi GOAP ya da BT çözümünü takabilir. Eylem kataloğu, zihin sorguları ve olay üretimi aynı kalır. v0.1'de sadece arayüz ve varsayılan uygulama var; adaptörler v0.2 sonrasında.

5.6 Sonuç Kuralları

- Biçim: `ne_zaman <olay türü> + kriterler (5.9) → etkiler`.
- Örnek: "Biri NPC'nin kendisine yalan söylediğini öğrenirse: o kişiye `yalanını_yakaladım(−30, 30 gün)` modifier'ı, güven ekseni −10, öfke duygusu (şiddet 0.6, hedef: yalancı), hafızaya önemi yüksek bir kayıt."
- Kurallar JSON ile yazılır ve kod değiştirmeden genişletilebilir.
- Etkiler sadece Zihin ve Dünya'yı değiştiren yeni olaylar olarak kayda düşer. Her olay `KuralId` ve `NedenOlaylar` taşır, böylece her değişiklik izlenebilir kalır.
- Her kuralın `cooldown`, `tick_başına_en_fazla` ve isteğe bağlı `yay` koşulları vardır (bkz. 5.10).
- Doğrulayıcı, etkisi boş olan kuralı reddeder (Kismet'in tuzağı).

5.7 Diyalog

- Karar modülü bir **niyet** üretir. Örnek: `{konuşan: Ayşe, niyet: uyar, konu: borç(Mehmet), ton: gergin, fikir_bandı: soğuk, gerekçe: [karar izinden ilk 2 ölçüt]}`.
- Şablonlar da **aynı kriter diliyle** seçilir. Her şablon bir `concept` (niyet) ve ağırlıklı kriterler taşır. En yüksek puanlı eşleşme kazanır, eşitlikte seed'li seçim yapılır. Genel bir varsayılan şablon her zaman bulunur; spesifik varyantlar üstüne eklenir (Valve Response System).
- Tekrar kontrolü: şablon grubu başına `tekrarsız`, `tekrar_bekleme(tick)`, `bir_kez`.
- Şablonlar hafızadaki gerçek olaylara gönderme yapabilir. Örnek: "Geçen pazar bana yardım ettin, ama bu başka mesele." Bu göndermeler, diyalog kriterlerinin hafızayı sorgulamasıyla mümkün olur: `hafıza.var(yardım_etti, oyuncu, son 7 gün)`.
- Varyasyon küçük bir gramer yapısıyla çoğaltılır (Tracery benzeri, kendi küçük uygulamamız; mevcut C# portları küçük ve bakımsız).
- Yapay zeka eklentisi için `IDialogueRenderer` arayüzü bırakılır. Eklenti metni yapay zekayla yazdırabilir, ama niyeti, içeriği ve gerekçeyi motor belirler. Akış tek yönlüdür; üretilen metin simülasyona geri yazılmaz. Arayüz, eklentiye seçilmiş gerçekleri ve gerekçeyi verir. Bunun sebebi, bağlamsız prompt'ların karakterleri birbirine benzetmesi, aşırı yüklü olanların ise olay uydurması (Slice of Life, 2025).
- Diyalog, durumu okumakla kalmaz; bir şablon `sonra: <hedef NPC> <concept>` ile başka bir NPC'nin cevabını tetikleyebilir (Valve'daki followup). Bu v0.1'de tek seviyeyle sınırlı.

5.8 Dış API

- `Submit(PlayerAction)`: oyuncu eylemini kayda ekler.
- `Tick(n)`: simülasyonu n adım ilerletir.

- Sorgular: NPC durumu, inançlar ve kanıtları, fikirler ve modifier'lar, stres, karar izi, `Neden(olayId)` (nedensellik zinciri).
- Dışarı akan olaylar: oyun bunlara abone olup sahneyi günceller.
- Kayıt ve yükleme: seed + senaryo + eylem günlüğü (JSON). Golden replay dosyalarıyla aynı biçimde.
- Örnek kullanım (README'deki hızlı başlangıcın taslağı):

```
var world = Scenario.Load("samples/KasabaHani/scenario.json", seed: 42);
world.Submit(PlayerAction.Tell(to: "Ayse", fact: "borclu(Mehmet, Ayse, 500)"));
world.Tick(3);
var can = world.Npc("Can");
Console.WriteLine(can.OpinionOf("player"));          // -22 (soğuk)
foreach (var line in world.Why(can.LastChange))      // nedensellik zinciri
    Console.WriteLine(line);
```

5.9 Ortak kriter dili (yeni)

Sonuç kuralları, diyalog şablonları ve eylem ön koşulları aynı yapıyı kullanır:

```
{ "alan": "fikir(konuşan, dinleyen)", "op": "<", "değer": -10, "ağırlık": 2 }
{ "alan": "inanç(dinleyen, borçlu(Mehmet,*,*))", "op": "güven>=", "değer": 0.5, "zorunlu": true }
{ "alan": "yay.kasaba_gerginliği", "op": "aralık", "değer": [2, 4] }
```

- Alanlar, zihin, dünya, hafıza ve yay değişkenleri üzerinde tanımlı sorgu fonksiyonlarıdır. v0.1'de sabit ve belgelenmiş küçük bir set var, yaklaşık 15 fonksiyon.
- Puan: tutan kriterlerin ağırlık toplamı. `zorunlu` bir kriter tutmazsa aday elenir.
- Tek ayrıştırıcı, tek doğrulayıcı ve tek kapsama aracı her üç kullanım için de çalışır.

5.10 Frenler ve korumalar (yeni)

- **Cooldown:** Kural başına, isteğe bağlı olarak (kural, NPC) çifti başına.
- **Tetiklenme sınırı:** Kural başına tick'te en fazla k kez. Ayrıca genel bir "tick başına en fazla M sonuç olayı" sınırı.
- **Zincir derinliği:** Bir oyuncu eyleminden doğan nedensellik zinciri aynı tick içinde en fazla D derinliğe iner (varsayılan 4). Kalan etkiler bir sonraki tick'e ertelenir. Bu, v1'deki "tur sınırı"nın yerine geçer.
- **Korumalı bayraklar:** Senaryo yazarı, hikaye için kritik varlıkları korur. Korumayı ihlal eden eylemler aday listesine hiç girmez.
- **Yay değişkenleri:** Senaryo düzeyinde birkaç sayısal değişken (`kasaba_gerginliği` gibi). Sonuç kuralları bunları hem okuyabilir hem değiştirebilir. Bu, yazarın simülasyonu kapatmadan dramatik yayı yönlendirmesini sağlar.
- Her fren tetiklendiğinde bir `fren` olayı kayda düşer. Böylece "bu neden olmadı?" sorusu da cevaplanabilir.

6. Tick döngüsü (bir adımda olanlar)

1. Bekleyen oyuncu eylemleri olay olarak kayda eklenir (`KuralId: player`).
2. Algı: yeni olayları kimin algıladığı hesaplanır, inançlara kanıt eklenir.
3. Sonuç kuralları çalışır (cooldown, sınır ve korumalar uygulanarak). Zihin değişiklikleri yeni olaylar olarak kayda düşer.
4. Süresi dolan modifier'lar düşer, ruh hali ve stres söner, hafıza silikleşir. Stres eşiği aşılırsa kriz olayı üretilir.
5. Her NPC için (sabit sırayla, ama anahtarlı RNG ile) `IActionSelector` bir eylem seçer. Korumalı varlıkları ihlal eden adaylar baştan elenir.
6. Seçilen eylemler olay olarak kayda düşer ve 2. adıma geri beslenir. Zincir derinliği D'yi aşarsa kalan etkiler bir sonraki tick'e ertelenir.
7. Durum hash'i alınır, dışarıya olaylar yayınlanır.

7. Konuşma sırası etkisi: örnek senaryo

1. Oyuncu Ayşe'ye "Mehmet borcunu ödemedi" der. Bu bir olaydır. Ayşe'nin inancına `duydu(oyuncu, güç 0.7)` kanıtı eklenir.
2. Bir sonraki tick'te Ayşe'nin sosyallik puanı yüksek olduğu ve konu dikkat çekici olduğu için, "Mehmet'in kardeşi Can'a anlat" eylemi ilk 3 aday arasında en yüksek puanı alır ve seçilir.
3. **Durum 1:** Oyuncu Can'la bu tick'ten önce konuşur. Can'ın fikri nötrdür; şablon havuzundan sakin ve yardımsever bir satır seçilir.
4. **Durum 2:** Oyuncu Can'la bu tick'ten sonra konuşur. Can'da `aileme_iftira(oyuncu)` modifier'ı (−22, 18 gün) vardır ve fikir bandı "soğuk"tur. Şablon "Ağabeyim hakkında ne dediğini Ayşe'den duydum." olur. Hata ayıklama ekranı zinciri

gösterir: oyuncu → Ayşe'ye anlattı (tick 40) → Ayşe Can'a anlattı (tick 41) → kural `aile_iftirası` → modifier.

5. **Durum 2b (stres):** Can'ın merhameti yüksektir. Oyuncu ondan Mehmet'i ele vermesini isterse Can bunu yapabilir, ama stresi artar. Birkaç olay sonra kriz eşiğinde Mehmet'e gidip itiraf edebilir. Bu da yeni bir olay zinciri başlatır.

Bu sahneler için hiçbir yazar dal yazmaz. Fark tamamen simülasyondan doğar. Bu senaryo, 3 sıra etkisi testinden biri ve tek seçim farkı raporunun ana örneği olur.

8. Araçlar

- **Konsol sandbox (CLI, `yanki`):** `run` (senaryo yükle, eylem gönder, tick ilerlet), `query` (durum sorgula), `why <olayId>` (nedensellik zinciri), `replay <dosya>` (tekrar oynat ve hash'i doğrula).
- **Senaryo doğrulayıcı (`yanki validate`):** JSON şeması, kriter dili sözdizimi, etkisi boş kural (hata), hiçbir ölçütte ya da kriterde kullanılmayan kişilik özelliği (uyarı), hiç ulaşılamayacak şablon (uyarı). Hata mesajları dosya ve satır gösterir.
- **Kural kapsama aracı (`yanki coverage --seeds 20`):** Toplu koşuda her kuralın ve şablonun tetiklenme sayısını listeler; hiç tetiklenmeyenleri ve çok sık tetiklenenleri işaretler. Yusuf'un denge çalışmasının ana aracı.
- **Ayrışma raporu (`yanki diff a.json b.json`):** Tek bir oyuncu kararı farklı olan iki replay'i karşılaştırır. Tick tick ayrışan olay, inanç ve diyalog sayılarını Markdown tablo olarak verir. Dedikodu ve stres kapatılarak ablasyon yapılabilir (`--ablate gossip,stress`).
- **Tarayıcı demosu + zihin paneli (Blazor WebAssembly):** Solda oyun (NPC listesi, konuşma, eylem menüsü), sağda panel. Panelde seçili NPC'nin kişiliği, fikirleri ve modifier'ları, stresi, inançları ve kanıtları, son karar izi ve "neden?" zinciri var. Zaman çizelgesinde bir tick'e geri sarma, günlüğün o tick'e kadar yeniden oynatılmasıyla yapılır. Replay dosyası indirilip yüklenebilir. İlişki grafiği v0.1'de basit bir tablo; görsel graf v0.2'de.

9. Test stratejisi

- **Birim testleri (xUnit):** Her modül ve kriter dili tek başına test edilir. CI her iki hedefte de çalışır (`netstandard2.1` testleri için `net10.0` test projesi + Unity uyumluluk derlemesi).
- **Senaryo testleri:** "Şu olay dizisi → şu sonuç" biçiminde yazılır. 3 sıra etkisi testi bunlara dahildir.
- **Golden replay testleri:** `tests/golden/` altında 3–5 kayıtlı oturum (seed + senaryo + eylem günlüğü + beklenen son hash). CI bunları tekrar oynatıp hash'i doğrular. Hash bilerek değiştiğinde güncelleme tek komutla yapılır ve PR'da gerekçesi yazılır.
- **Özellik tabanlı testler (CsCheck):** Rastgele eylem dizileriyle şu değişmezler kontrol edilir:
- eksenler ve stres sınır içinde kalır,
- aynı seed ve girdi her zaman aynı hash'i verir,
- "N tick koş, kaydet, yükle, M tick koş" ile "N+M tick kesintisiz koş" aynı sonucu verir,
- korumalı bir varlık hiçbir eylem dizisinde ihlal edilmez,
- her olayın `NedenOlaylar` kimlikleri günlükte vardır.

Başarısız örneğin seed'i test çıktısına yazılır.

- **Determinizm kuralları:** Simülasyon kodunda `System.Random`, `DateTime.Now` ve sırasız `Dictionary` gezintisi yasak. Bunu küçük bir analyzer ya da test denetler. Tek RNG kaynağı var, anahtarlı.
- **Platformlar arası hash:** Bir golden replay, Blazor WASM içinde de koşturulur ve aynı hash beklenir (Playwright ile tek bir uçtan uca test; zor çıkarsa manuel kontrol ve v0.2).
- **Performans (BenchmarkDotNet):** 10 / 50 / 200 NPC için tick süresi ve `[MemoryDiagnoser]` ile tick başına bellek tahsisi. Sonuç tablosu README'de; donanım ve .NET sürümü belirtilir.
- **Değerlendirme (test değil, rapor):** Tek seçim farkı ayrışma raporu, ablasyon (dedikodu kapalı, stres kapalı) ve 3–5 kişilik kör playtest. Playtest'te aynı senaryo "tam sistem" ve "ablasyonlu" sürümle oynatılır, oyuncu hangisinde NPC'lerin daha tutarlı ve tepkili olduğunu söyler. Bu küçük ve bilimsel olmayan bir deneme; sonuçlar devlog'da dürüstçe paylaşılır.
- **CI:** Her push'ta GitHub Actions üzerinden derleme, testler, golden replay'ler ve coverage (codecov) çalışır. Benchmark'lar elle ya da etiketle tetiklenir.

10. Demo senaryosu

Motoru göstermek için küçük bir **"Kasaba Hanı"** senaryosu kurulur. Tarayıcıda 5–10 dakikada oynanabilir.

- 8 NPC (en fazla 10): hancı, borçlu çiftçi, dedikoducu terzi, şüpheli yabancı, muhtar, çiftçinin kardeşi Can ve diğerleri.

Her NPC dosyası en fazla yaklaşık 10 alan içerir.

- 3 gerilim tohumu: bir borç, kayıp bir eşya, bir hırsızlık. Gizli ilişki v0.2'ye kaldı.
- İçerik bütçesi (v0.1): yaklaşık 15 eylem, 25–35 sonuç kuralı, 60–80 şablon (her niyet için 1 genel + birkaç spesifik varyant), 2 yay değişkeni, 1–2 korumalı varlık.
- Oyuncu NPC'lerle konuşur, bilgi verir ya da saklar, iyilik ya da kötülük yapar. Bir hafta (oyun içi) sonunda kasabanın durumu oyuncunun kararlarına göre şekillenir. Son ekranda bir "kasaba özeti" (bu hafta neler oldu, kim kime küs) ve "bu oturumun replay dosyası" çıkar.
- Her kişilik özelliği en az bir eylem ölçütünde **ve** en az bir diyalog kriterinde kullanılır. Doğrulayıcı bunu denetler; mekaniği olmayan "süs" özellik olmaz.

Bu senaryo, ileride yapılacak dedektif oyunundan bağımsızdır. Sadece motorun yeteneklerini göstermek içindir. Kanıt listeli inançlar, "kimden duydu" sorgusu ve korumalı bayraklar ise dedektif oyununun doğrudan altyapısıdır.

11. Rol dağılımı

Alan	Yusuf	Claude Code
Sistem tasarımı ve kurallar	Karar verir, onaylar, ADR'nin "neden" kısmını yazar	Seçenek ve taslak önerir, alternatifleri listeler
Kişilik / ilişki / stres modeli, denge sayıları	Belirler, coverage ve diff araçlarıyla deneyerek ayarlar	Ayar araçlarını yazar
Kriter dili sözlüğü (hangi sorgu fonksiyonları var)	Belirler	Ayrıştırıcı, doğrulayıcı ve testleri yazar
Demo senaryosu, NPC'ler, şablon metinleri (Türkçe ve İngilizce)	Yazar	Biçim, doğrulama, eksik varyant raporu
Kod	Okur, anlar, PR olarak inceler	Yazar
Testler	Senaryoları, değişmezleri ve golden oturumları tanımlar	Test kodunu yazar
Blazor demo arayüzü	Ekran akışını ve neyin gösterileceğini belirler	Yazar
Playtest	Düzenler, gözlemler, notları yazar	Sonuç tablosunu biçimlendirir
Dokümantasyon ve devlog	ADR gerekçeleri, devlog yazıları, README tanıtım metni	Biçim, taslak, API dokümanı

Yapay zeka kullanımının belgelenmesi:

- Repoda bir `CLAUDE.md` dosyası bulunur.
- `docs/ai-usage.md`: hangi araç kullanıldı (Claude Code), hangi kısımları yapay zeka taslakladı, hangilerini Yusuf tasarladı, çıktı nasıl doğrulandı (testler, golden replay'ler, benchmark'lar, PR incelemesi). Özür dilemeden, açıkça.
- Önemli kararlar `docs/adr/` altında ADR olarak tutulur. Her ADR reddedilen alternatifleri de içerir ve Yusuf'un kendi cümleleriyle yazılır.
- Tek kişilik projede bile iş PR üzerinden ilerler. Commit'ler küçük ve anlamlıdır; yapay zekanın yazdığı commit'ler `Co-Authored-By` satırı taşır.
- Her kilometre taşında bir devlog yazısı: ne denendi, ne bozuldu, ne değişti.
- README'de "Bu proje nasıl geliştirildi" bölümü ve 3–5 dakikalık anlatımlı bir walkthrough videosu.
- Yusuf her modülü kendi cümleleriyle anlatabilecek kadar anlamalıdır. Mülakat için hazır olması gerekenler: tick döngüsünü tahtada çizmek, determinizm stratejisini anlatmak ve çözdüğü bir zor hatayı anlatmak. Her modül sonrası `docs/notes/` altına kısa, kendi anlatımıyla bir not yazar.

12. 12 haftalık plan

Varsayım: haftada 12–15 saat (toplam yaklaşık 150–180 saat), okul ve YKS hazırlığı paralel yürüyor. Başlangıç 2 Kasım 2026, bitiş 24 Ocak 2027. 9. hafta (yılbaşı) ve 11. hafta (muhtemel final dönemi) bilinçli olarak hafif tutuldu. Saat sütunu Yusuf'un kendi zamanı; Claude Code'un yazdığı kodun incelemesi dahil.

Hafta	Tarih	Odak	Teslim edilen	Yusuf (saat)
1	2–8 Kas	Kurulum ve iskelet	Repo (MIT), .NET çözümü (`netstandard2.1;net10.0`), CI, CLAUDE.md, README iskeleti ve rozetler, `ai-usage.md`, ADR-001..006, boş modüller	12
2	9–15 Kas	Olay kaydı, dünya, kriter dili	Olay tipleri (`NedenOlaylar`, `KuralId`), tek değerli gerçekler, mekan grafiği, anahtarlı RNG, durum hash'i, kriter ayrıştırıcı ve puanlayıcı, testler	14

3	16–22 Kas	Zihin ve doğrudan algı	Kişilik, 3 eksen + fikir modifier'ları, tek duygu ruh hali, kanıt listeli inançlar, hafıza, sönümleme; görünürlüklü doğrudan algı	15
4	23–29 Kas	Sonuç kuralları ve **ince dikey dilim**	Kural motoru (kriter diliyle), doğrulayıcı v0, tick döngüsü v0, minimal CLI; 3 NPC'lik mini senaryo uçtan uca; golden replay #1 CI'da; devlog #1	14
5	30 Kas–6 Ara	Utility AI	Eylem tanımları (JSON), eğriler, çarpan aralıkları, top-N seçim, karar izi, `IActionSelector` + `UtilitySelector`, `yanki why`	15
6	7–13 Ara	Dedikodu ve stres	"Anlat" eylemi, kanıt aktarımı, top-n konu sınırı, yalan kaydı; stres, kriz olayları	15
7	14–20 Ara	Frenler ve sağlamlık	Cooldown, tetiklenme ve zincir derinliği sınırları, korumalı bayraklar, yay değişkenleri; 3 sıra etkisi testi; CsCheck değişmezleri; kayıt/yükleme (günlük tabanlı); devlog #2	14
8	21–27 Ara	Diyalog	Niyet, kriter tabanlı şablon seçimi, tekrar kontrolleri, hafıza göndermeleri, mini gramer, tek seviyeli followup, `IDialogueRenderer`	14
9	28 Ara–3 Oca	Demo içeriği (hafif hafta, tampon)	Kasaba Hanı: 8 NPC, eylemler, kurallar, şablonlar (Yusuf ağırlıklı yazım); `yanki validate` tamamlanır; `yanki coverage`	10
10	4–10 Oca	Tarayıcı demosu	Blazor WASM: oyun ekranı + zihin paneli + "neden?" zinciri + tick'e geri sarma + replay indir/yükle; GitHub Pages yayını	14
11	11–17 Oca	Ölçüm ve denge (hafif hafta, finaller)	BenchmarkDotNet tablosu, `yanki diff` ve ablasyon raporu, coverage'a göre denge turu, 3–5 kişilik kör playtest	10
12	18–24 Oca	Cila ve yayın	README (GIF, demo linki, hızlı başlangıç, Mermaid diyagram, benchmark, karşılaştırma tablosu), docs sitesi, walkthrough videosu, CHANGELOG, tag'le NuGet `v0.1.0`, devlog #3	14

Toplam yaklaşık 161 saat. Bu, 12–15 saatlik varsayımın ortası; böylece bir haftalık kaymayı emecek pay kalıyor.

Kontrol noktaları:

- **4. hafta sonu (29 Kas):** İnce dikey dilim çalışıyor. Oyuncu eylemi → algı → kural → fikir değişimi → `why` zinciri → CI'da golden replay. Bu olmadıysa 6. haftadaki stres ve 8. haftadaki followup kesilir, sıra kaydırılır.
- **7. hafta sonu (20 Ara):** Simülasyon tam. Karar, dedikodu, stres ve frenler deterministik çalışıyor; 3 sıra etkisi testi ve CsCheck değişmezleri yeşil. Olmadıysa diyalog sadeleşir (gramer ve followup v0.2'ye gider).
- **10. hafta sonu (10 Oca):** Demo tarayıcıda baştan sona oynanabiliyor. Olmadıysa 11. haftadaki playtest ve ablasyon küçültülür, demo önceliklidir.
- **12. hafta sonu (24 Oca):** v0.1.0 yayında. Demo, README, docs ve NuGet hazır.

Kesme sırası (gerekirse, ilk kesilen başta): platformlar arası hash'in Playwright testi → tek seviyeli followup → mini gramer (düz şablonla devam) → kör playtest (sadece ayrışma raporu kalır) → docs sitesi (README + `docs/` klasörü yeterli). Demo, golden replay'ler ve `why` zinciri **kesilmez**; projenin vitrini bunlar.

13. Repo yapısı (taslak)

```
yanki/
├─ src/
│  ├─ Yanki.Core/              # olay kaydı, dünya, zihin, algı, karar, kurallar, kriter dili, frenler
│  │                           # netstandard2.1;net10.0, bağımlılıksız
│  ├─ Yanki.Dialogue/          # şablon seçimi, gramer, IDialogueRenderer
│  ├─ Yanki.Cli/               # yanki run / why / replay / validate / coverage / diff
│  └─ Yanki.Demo.Web/          # Blazor WebAssembly: Kasaba Hanı + zihin paneli
├─ tests/
│  ├─ Yanki.Core.Tests/        # xUnit + CsCheck
│  ├─ Yanki.Scenarios.Tests/   # senaryo ve sıra etkisi testleri
│  ├─ golden/                  # kayıtlı oturumlar + beklenen hash'ler
│  └─ Yanki.Benchmarks/        # BenchmarkDotNet
├─ samples/
│  ├─ KasabaHani/              # demo senaryosu (JSON)
│  └─ Quickstart/              # README'deki 15 satırlık konsol örneği
├─ docs/
│  ├─ architecture.md          # Mermaid diyagramlı mimari özeti
│  ├─ criteria-language.md     # kriter dili referansı
│  ├─ authoring-guide.md       # senaryo yazım rehberi
│  ├─ ai-usage.md
│  ├─ evaluation.md            # ayrışma raporu, ablasyon, playtest sonuçları
│  ├─ notes/                   # Yusuf'un modül notları
│  ├─ devlog/
│  └─ adr/
├─ .github/workflows/          # ci.yml, release.yml (tag → NuGet), pages.yml (demo + docs)
├─ CHANGELOG.md
├─ CONTRIBUTING.md
├─ LICENSE                     # MIT
├─ CLAUDE.md
└─ README.md
```

README iskeleti (1. haftada boş başlıklarla kurulur, 12. haftada doldurulur):

1. Rozetler: CI, coverage, NuGet, lisans, .NET hedefleri, "▶ Demoyu oyna".
2. Hero GIF (10–15 sn): oyuncu bir şey söylüyor, panelde bir NPC'nin fikri değişiyor, "neden?" zinciri açılıyor.
3. Tek cümlelik tanım + 5 madde: Deterministik / Motordan bağımsız / Açıklanabilir / Test edilmiş / Hızlı. Her madde kanıtına link verir.
4. Demo linki ve 60–90 sn'lik video.
5. Kurulum (`dotnet add package Yanki.Core`) ve 15 satırlık hızlı başlangıç.
6. Temel kavramlar (`<details>` içinde): zihin katmanları, kanıtlı inançlar, kriter dili, frenler.
7. Mermaid mimari diyagramı + `docs/architecture.md` linki.
8. "Seçimler önemli mi?": ayrışma raporu tablosu.
9. Benchmark tablosu.
10. Benzer sistemlerle karşılaştırma (Love/Hate, Ensemble, Yarn/ink ile ilişkisi).
11. Bu proje nasıl geliştirildi (yapay zeka kullanımı) + ADR ve devlog linkleri.
12. Yol haritası (v0.2 sonrası), katkı, lisans.

14. Riskler ve önlemler

Risk	Etki	Önlem
Kapsam büyür	Proje bitmez	Kapsam dışı ve "v0.2 sonrası" listelerine sadık kalmak; 4/7/10. hafta kontrol noktaları; 12. bölümdeki kesme sırası
"Oyunu olmayan motor" algısı	Portfolyo etkisi zayıflar	Tarayıcı demosu README'nin ilk ekranında; ayrışma raporu; hero GIF
NPC'ler mekanik hissettirir	Asıl vaat zayıflar	4. haftada dikey dilim; top-N seçim; stres krizleri; hafıza göndermeli şablonlar; kör playtest
Bilgi yayılımı patlar (herkes her şeyi bilir)	Sonuçlar anlamsızlaşır	Top-n konu sınırı, aktarımda kanıt gücü düşüşü, anlatma eylemine ilgi eşiği; ablasyonla ölçmek
Kaskad ve sarmallar	Kasaba birkaç tick'te çöker	Cooldown, tetiklenme ve zincir derinliği sınırları; `fren` olayları; CsCheck ile rastgele dizilerde test
NPC'ler hikayeyi bozar (Oblivion sorunu)	Senaryo kontrol edilemez	Korumalı bayraklar, yay değişkenleri; korumanın ihlal edilmediğini gösteren özellik testi
Yazım ve ayar yükü ağır	Diyalog tekrar eder, denge tutmaz	Minimal şema, içerik bütçesi, genel + spesifik şablon yapısı, doğrulayıcı uyarıları, `coverage` aracı
Determinizm bozulur (WASM dahil)	Testler ve tekrar oynatma çöker	Anahtarlı tek RNG, sıralı koleksiyonlar, yasaklı API denetimi, hash'te sabit hassasiyet, golden replay'ler
Unity uyumu bozulur	"Motordan bağımsız"	`netstandard2.1` hedefi CI'da derlenir; net10'a özgü API kullanılmaz; v0.2'de Unity

	iddiası çürür	örneği
Kod anlaşılmadan yazılır	Mülakatta zarar	PR üzerinden inceleme, her modül sonrası `docs/notes/` notu, ADR'ler, walkthrough videosu
Okul, YKS ve finallerle çakışma	Gecikme	9. ve 11. hafta hafif; yaklaşık 161 saatlik plan, 180 saatlik üst sınırın altında; kesme sırası hazır

15. Sonraki adım

1. Bu plan onaylanınca, 2 Kasım'dan önce ADR-001..006 taslakları çıkarılır: olay kaynaklı çekirdek, utility + `IActionSelector`, kriter dili, hedef framework'ler, Blazor demo kararı, MIT lisansı.
2. Yusuf, 1. haftada kriter dili sözlüğünün ilk listesini (yaklaşık 15 sorgu fonksiyonu) ve Kasaba Hanı'nın 8 NPC'sinin tek satırlık tanımlarını yazar.
3. 4. hafta kontrol noktasının sonucu, plan v3 gerekip gerekmediğine karar verir.

16. v0.2 sonrası (bilinçli olarak ertelenenler)

Öncelik	Madde	Neden ertelendi / neden değerli
1	Küçük Unity örnek projesi + OpenUPM paketi	Türk mobil stüdyoları Unity tecrübesini artı sayıyor; 12 haftaya sığmadı
2	Yarn Spinner köprüsü (`Yanki.Yarn` ayrı paket)	Zihin durumunu Yarn değişkenleri olarak sunmak; çekirdeğe bağımlılık sokmamak için ayrı paket
3	Anlık görüntü (snapshot)	10–50 NPC için günlükten yeniden oynatma yeterli; uzun oturumlarda gerekecek
4	Hikaye ayıklama sorguları (Winnow tarzı)	"İhanet", "söylenti kaynağına ulaştı" gibi anları günlükte bulmak; dedektif oyunu için değerli
5	İnanç bozulması (mutation) ve aktif bilgi arama	Talk of the Town'ın kalan özellikleri
6	Versu tarzı "eylemi simüle et, sonucu puanla" (derinlik 1)	Puanları tahminden gerçek sonuca taşır; maliyeti benchmark'la ölçülmeli
7	Görsel ilişki grafiği ve kural bağımlılık görselleştirmesi	Hades'in eksikliği; v0.1'de tablo yeterli
8	GOAP adaptörü (örn. MountainGoap) `IActionSelector` üzerinden	Arayüzün gerçekten takılabilir olduğunu kanıtlar
9	Yapay zeka renderer örneği (`IDialogueRenderer`), beyaz listeli tek yönlü akışla	Seçilmiş gerçekler + gerekçe ile prompt; uydurmaya karşı test
10	Kalıcı anıların kişiliği çok yavaş kaydırması (Dwarf Fortress)	v0.1'de kişilik sabit; kayma tutarlılık testlerini zorlaştırır
11	Gizli ilişki tohumu, ikinci demo senaryosu	İçerik bütçesi
12	Güncelleme sıklığı / LOD ve 1.000+ NPC benchmark'ı	AI programcısı ilanlarındaki ölçeklenebilirlik sinyali; hedef ölçek dışında